Linked List

Data Structure: LinkedList

A linked list is a data structure used for storing collections of data.

A linked list has the following properties.

- Successive elements are connected by pointers
- The last element points to NULL
- Can grow or shrink in size during execution of a program
- Can be made just as long as required (until systems memory exhausts)
- Does not waste memory space (but takes some extra memory for pointers). It allocates memory as list grows.

Comparison of Linked Lists with Arrays & Dynamic Arrays

Parameter	Linked List	Array	Dynamic Array
Indexing	$O(n)$	$O(1)$	$O(1)$
Insertion/deletion at beginning	$O(1)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Insertion at ending	$O(n)$	$O(1)$, if array is not full	$O(1)$, if array is not full $O(n)$, if array is full
Deletion at ending	$O(n)$	$O(1)$	$O(n)$
Insertion in middle	$O(n)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Deletion in middle	$O(n)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Wasted space	$O(n)$ (for pointers)	0	~~$O(n)$~~ 0

LinkedList ADT

The following operations make linked lists an ADT. (For simplicity, assume the data is an integer type.)

Main Linked Lists Operations

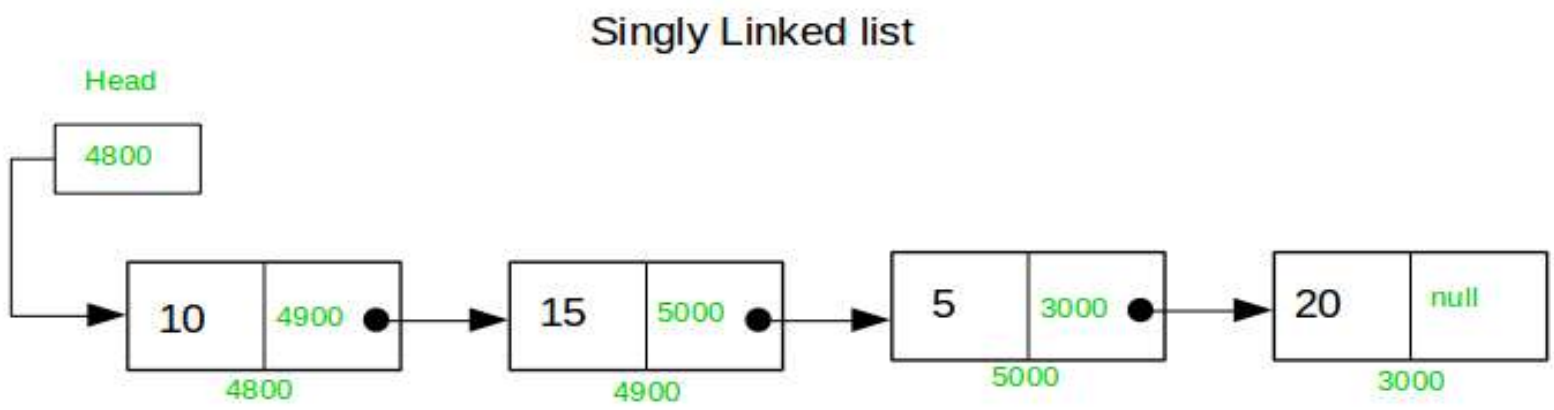
- Insert: inserts an element into the list
- Delete: removes and returns the specified position element from the list

Auxiliary Linked Lists Operations

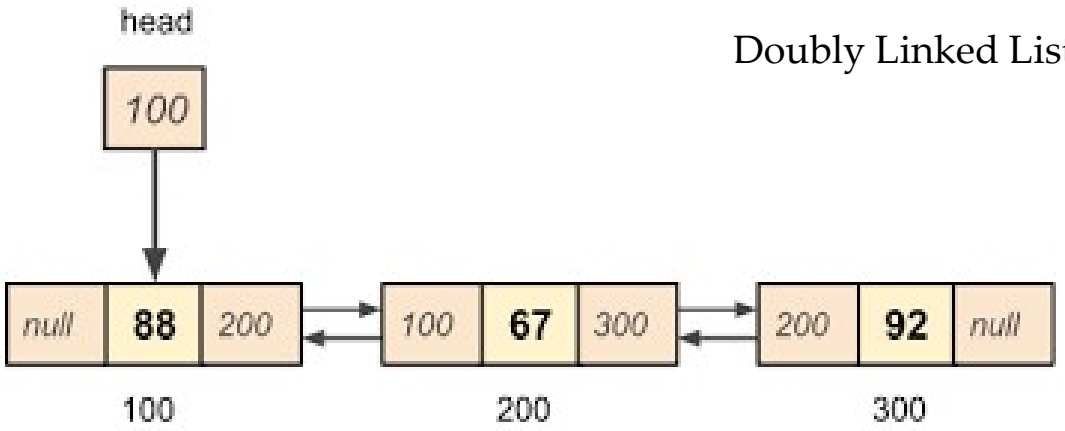
- Delete List: removes all elements of the list (disposes the list) *(Java ??)*
- List traversal: visiting all nodes of LinkedList
- Count: returns the number of elements in the list
- Find n th node from the end of the list

Types of LinkedList:

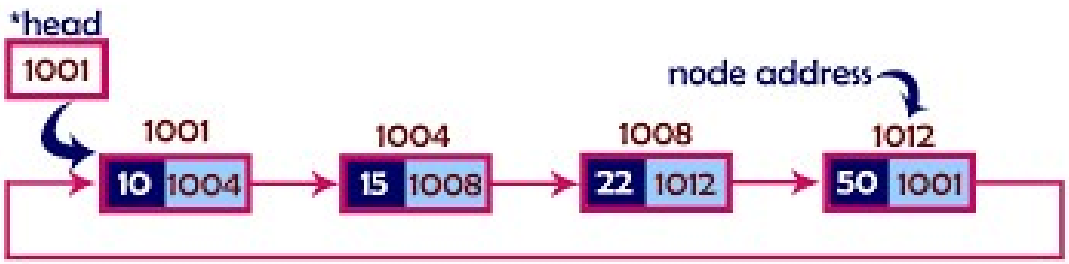
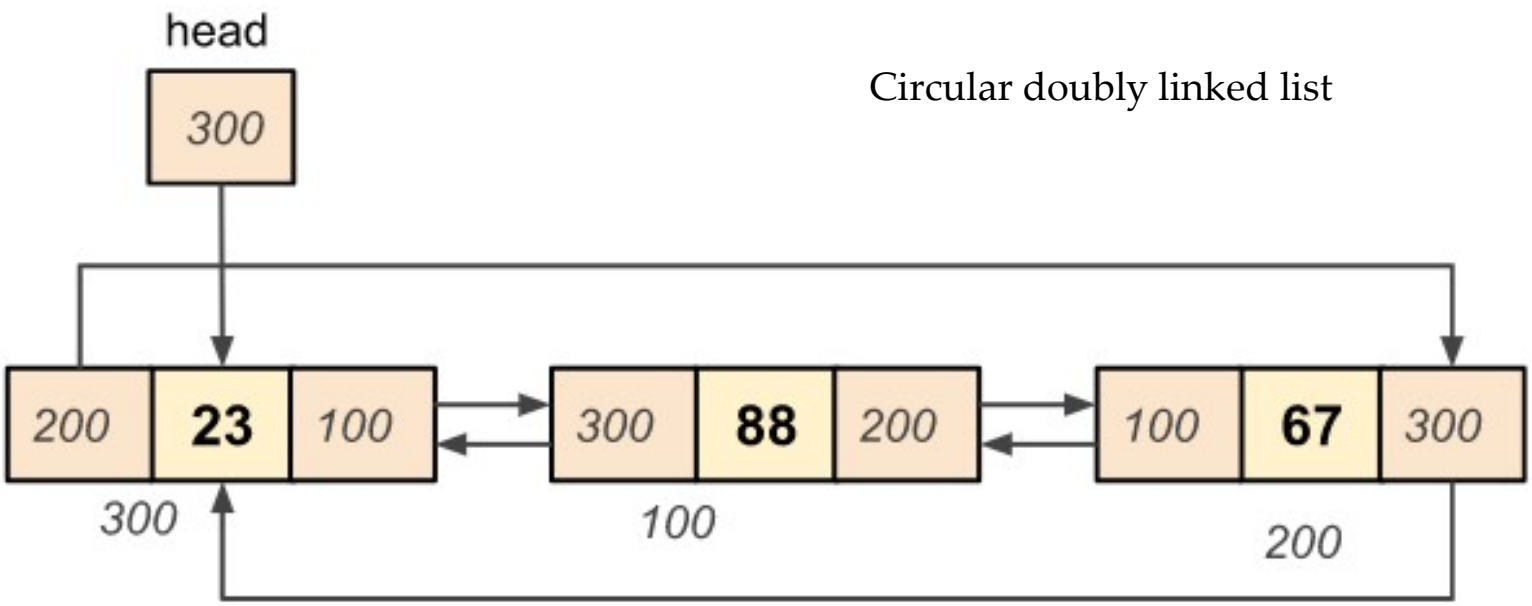
1. Singly
2. Doubly
3. Circular
4. Doubly circular



Doubly Linked List



Circular Linked List



Circular doubly linked list